

Can Crowdsourced Reviews Predict Food Safety? A Three-Model Investigation of NC Restaurant Inspection Grades

Roshan Gill, Jonas Neves, Dominic Tanzillo
Ensemble Method
AIPi 540 Deep Learning
Duke University, Spring 2026

Abstract

Restaurant health inspection grades are public record, yet consumers rarely consult them, relying instead on crowdsourced review platforms such as Google to choose where to eat. We investigate whether the language and metadata of public reviews contain signal that predicts official food safety outcomes. Using 231,160 NC DHHS inspection records across 31,799 restaurants linked to Google Places review data, we train and compare four models: a majority-class baseline, a Random Forest on six structured review features, a Random Forest augmented with TF-IDF text features, and a fine-tuned DistilBERT classifier on raw review text. On a binary task (A vs. Flagged), the best models achieve macro F1 of 0.57–0.58, up from 0.50 for the uninformative baseline. A threshold sweep over inspection scores reveals that at a <98 cutoff the Random Forest reaches macro F1 = 0.76 with 85% sensitivity. Two silent data pipeline bugs (case-sensitive fuzzy matching and a UTF-8 BOM encoding error) nearly made this learnable problem appear unlearnable; fixing them increased flagged training samples from 197 to 3,354 (17 $\times$). The deployed web application provides per-restaurant risk assessments with SHAP explanations. Our findings suggest that review metadata carries measurable but limited safety signal, primarily through volume and length proxies rather than safety-specific language.

1 Introduction

North Carolina’s Department of Health and Human Services (DHHS) inspects every licensed food establishment and assigns letter grades (A, B, or C) based on sanitation, temperature control, pest management, and food handling. These grades are public record, but consumers rarely check them. Instead, they rely on crowdsourced review platforms like Google to decide where to eat, where users describe taste and service, not sanitizer concentration or temperature logs.

This project asks: **does the language and sentiment of public restaurant reviews contain signal that predicts official food safety inspection outcomes?** If so, a predictive model could surface restaurants where high public ratings mask poor inspection records, exposing the gap between public perception and regulatory reality.

We frame this as a binary classification problem: **A (safe) vs. Flagged (B or C inspection grade)**, trained on 231,160 NC DHHS inspection records linked to Google Places review data. The dataset exhibits extreme class imbalance: 98.5% of inspections receive grade A, yielding a 68:1 safe-to-flagged ratio. A wrong prediction (labeling a flagged kitchen as safe) means someone eats there trusting a model that said it was safe.

Our contributions are:

1. **Multi-source data pipeline.** We link 231,160 NC DHHS inspection records to 17,561 Google Places listings via fuzzy matching, creating the first NC-specific inspection/review dataset.
2. **Four-model comparison.** We evaluate a majority-class baseline, a Random Forest on structured features, a Random Forest augmented with TF-IDF, and a fine-tuned DistilBERT, spanning from naive to deep learning approaches on the same binary task.
3. **Data quality experiment.** We document how two silent pipeline bugs made a learnable problem appear unlearnable, demonstrating that data auditing is a prerequisite for model evaluation.
4. **Deployed screening tool.** A live web application provides per-restaurant risk assessments with SHAP feature attributions, demonstrating the UX pattern for a review-based safety screening system.

2 Data Sources

Table 1: Data sources used in this study. NC DHHS records are public government data; Google Places data was accessed via the official API.

Source	Records	Access Method	License
NC DHHS Environmental Health	231,160	CDP portal, CSV export	Public record (G.S. 132-1)
Google Places API	17,561	googlemaps Python client	Google ToS

The NC DHHS data provides inspection date, establishment name, address, score (0–100), grade (A/B/C), and inspector ID across 100 counties and 31,799 restaurants (2020–2026). Each restaurant has an average of ~ 7 inspections over this period. Google Places data provides star rating, review count, and up to 5 review texts per listing.

Linking. Inspections and Google listings were joined on `state_id` after fuzzy name matching using `rapidfuzz.fuzz.token_sort_ratio` with `processor=default_process` for case normalization. A match threshold of 50 was used. Of 231,160 inspections, 111,542 (48.2%) were successfully linked to a Google listing with review data.

Class distribution. The dataset exhibits significant imbalance: 227,806 A (98.5%), 3,249 B (1.4%), 105 C (0.05%). We collapsed B and C into a single “Flagged” class ($n = 3,354$), creating a binary classification problem with a 68:1 imbalance ratio, far more extreme than prior restaurant safety prediction work [Kang et al., 2013].

3 Related Work

Prior work on predicting restaurant health outcomes from online data spans several methodological approaches.

Kang et al. [2013] used Yelp review text to predict hygiene violations in Seattle restaurants, reporting over 82% accuracy using unigram features. Altenburger and Ho [2019] later showed that extreme imbalanced sampling drove much of the reported accuracy. Their dataset had a more balanced violation distribution than NC’s heavily A-skewed grading system.

Sadilek et al. [2018], published in *npj Digital Medicine*, combined anonymized aggregated location history from opted-in users with illness-related search queries to identify potentially unsafe restaurants in Chicago and Las Vegas. Their approach relied on indirect behavioral signals rather than review text, achieving meaningful recall on serious violations.

Nsoesie et al. [2014], published in *Preventive Medicine*, compared the distribution of implicated food categories in Yelp illness-related reviews against CDC outbreak surveillance reports. They

found that Yelp data captured similar food category patterns to official reports, suggesting review platforms could complement (but not replace) traditional surveillance.

Our work differs in several ways: (1) we use NC’s letter-grade system rather than binary violation detection, (2) our class imbalance is far more extreme (98.5% A), (3) we compare four model architectures from naive baseline through deep learning, and (4) we explicitly test whether review language carries food safety signal at all, rather than assuming it does.

4 Evaluation Strategy and Metrics

Metric selection is critical in this domain because the 68:1 class imbalance renders several standard metrics uninformative. We considered and rejected several alternatives before selecting our evaluation metrics:

Why not accuracy? A classifier that predicts A for every restaurant achieves 98.5% accuracy while catching zero unsafe establishments. Accuracy is dominated by the majority class and conveys no information about the model’s ability to detect flagged restaurants.

Why not weighted F1? Weighted F1 weights each class’s F1 by its support (sample count). With 98.5% of samples in class A, weighted F1 is similarly dominated by majority-class performance and would report >0.98 for a model that predicts all-safe.

Why not AUC-ROC? AUC measures ranking quality across all thresholds. On extreme imbalance, AUC can appear high even when the model produces few true positives at any practical threshold. We report AUC where applicable but do not use it as the primary metric because it obscures the precision/recall trade-off at the specific operating point.

We evaluate on two complementary metrics:

Macro F1 (primary metric). The unweighted mean of per-class F1 scores. With two classes, macro F1 assigns equal importance to the safe and flagged classes regardless of their sample sizes. A majority-class baseline scores exactly 0.50 macro F1 (perfect F1 on class A, zero F1 on class Flagged, averaged). Any model that fails to improve on 0.50 has learned nothing beyond class frequency. We select macro F1 as the primary metric because it forces models to perform on the minority class, which is the only class that matters for safety screening. Concretely: a model with macro F1 = 0.57 must be detecting some flagged restaurants, while a model at 0.50 is not.

Flagged recall (safety metric). The fraction of truly B/C restaurants that the model correctly identifies. This is the safety-critical metric: a missed B/C grade means a consumer eats at a restaurant with documented sanitation failures, trusting a model that said it was safe. In a deployment context, false negatives are more dangerous than false positives, because the cost of missing a flagged restaurant (potential illness) exceeds the cost of over-flagging a safe one (unnecessary caution).

Together, these two metrics provide complementary views: macro F1 captures overall discrimination ability across both classes, while flagged recall isolates the safety-critical question of how many dangerous restaurants the model catches.

5 Data Processing Pipeline

The pipeline consists of four stages:

1. **Collection** (`scripts/make_dataset.py`). NC DHHS inspection records are scraped from the CDP portal via county-level CSV exports for years 2020–2026. Records are filtered to restaurant establishment types (codes 1, 2, 3, 4, 14, 15).
2. **Google linkage** (`scripts/make_dataset.py`). Each inspection record is matched to a Google Places listing using `find_place` with a combined name+address+city query. Match quality is scored using `fuzz.token_sort_ratio` with case normalization. Listings above threshold (50) are retained, and up to 5 reviews are fetched.
3. **Feature engineering** (`scripts/build_features.py`). Year-level inspection files are merged and deduplicated on (`state_id`, `inspection_date`). Google review data is

left-joined on `state_id`. Six features are engineered for the structured Random Forest (Table 2).

4. **Target encoding.** Grades are binarized: A = 0 (safe), B or C = 1 (flagged).

Table 2: Engineered features for the structured Random Forest.

Feature	Description	Type
<code>google_rating</code>	Google star rating (1.0–5.0)	Continuous
<code>google_review_count_log</code>	$\log(1 + \text{review count})$	Continuous
<code>review_word_count</code>	Total words in concatenated review text	Count
<code>review_avg_word_len</code>	Mean word length (complexity proxy)	Continuous
<code>safety_keyword_count</code>	Safety-adjacent terms (dirty, roach, sick, mold, ...)	Count
<code>negative_phrase_count</code>	Strongly negative phrases (terrible, food poisoning, ...)	Count

Data quality fixes. Two pipeline bugs silently degraded the training data: (1) case-sensitive fuzzy matching that reduced Google linkage from 14,868 to 432 matches, and (2) a BOM encoding error that dropped all inspection dates, collapsing 232K rows to 31K and losing 94% of B/C grade samples. Both fixes are documented as the project’s primary experiment (Section 10).

6 Hyperparameter Tuning Strategy

Random Forest. Tuned via 5-fold `GridSearchCV` optimizing `f1_macro`:

Table 3: Random Forest hyperparameter search space and selected values.

Hyperparameter	Search Space	Best Value
<code>n_estimators</code>	{100, 200}	100
<code>max_depth</code>	{None, 10, 20}	20
<code>min_samples_split</code>	{2, 5}	2

Balanced class weights were applied to counteract the 68:1 imbalance, effectively upweighting the minority class by the inverse class frequency. SMOTE oversampling was tested but produced no improvement over balanced weights alone.

DistilBERT. Trained for 3 epochs with batch size 16, max sequence length 256, per-epoch evaluation, and early stopping by lowest validation loss. Class-weighted cross-entropy loss was applied using inverse class frequencies. No learning rate sweep was performed due to computational constraints (T4 GPU via Google Colab).

7 Models Evaluated

7.1 Naive Baseline (DummyClassifier)

A majority-class predictor (`strategy='most_frequent'`) that always predicts A. This establishes the performance floor: any useful model must exceed macro F1 = 0.50. With 98.5% class A, a model that learns nothing scores extremely well on accuracy but contributes zero predictive value.

7.2 Random Forest on Structured Features

A Random Forest classifier trained on the 6 Google-derived features (Table 2) with balanced class weights. SHAP TreeExplainer [Lundberg and Lee, 2017] provides per-prediction feature attribution, surfaced in the web application.

Rationale. Random Forest [Breiman, 2001] handles tabular mixed-type features well and is robust to feature scale. The SHAP integration provides interpretable explanations, important for a consumer-facing application where users need to understand *why* a restaurant was flagged. Balanced class weights force the model to attend to the minority class.

7.3 Random Forest with TF-IDF Augmentation

The same Random Forest architecture with the 6 structured features augmented by TF-IDF features extracted from concatenated review text. This tests whether bag-of-words text representations improve performance over metadata alone.

Rationale. TF-IDF captures term-frequency patterns that the 6 engineered features may miss. If safety-relevant terms appear in reviews with frequencies that correlate with inspection outcomes, TF-IDF should surface them. This model bridges hand-crafted features and full language model approaches.

7.4 DistilBERT Fine-Tuned on Review Text

DistilBERT [Sanh et al., 2019] fine-tuned for binary sequence classification on concatenated Google review text. Trained on the 111,542 rows with review text on a T4 GPU via Google Colab, with class-weighted loss.

Rationale. If food safety signal exists in review text, a pretrained language model should find it. DistilBERT can capture semantic patterns (“the bathroom was filthy,” “saw a roach”) that keyword counting would miss. This model tests the upper bound of what NLP can extract from this data.

8 Results

8.1 Quantitative Comparison

Table 4 presents the main results on the held-out test set.

Table 4: Binary classification performance (A vs. Flagged) on the test set ($n = 46,232$). All trained models exceed the uninformative baseline (macro F1 > 0.50). DistilBERT was evaluated on a subset with review text ($n \approx 22K$).

Model	Macro F1	Flag. Precision	Flag. Recall	Flag. F1
Naive Baseline	0.496	0.00	0.00	0.00
Random Forest (structured)	0.568	0.11	0.28	0.16
Random Forest (struct + TF-IDF)	0.570	0.11	0.29	0.16
DistilBERT (class-weighted)	0.580	—	—	—

All three trained models improve over the uninformative baseline (macro F1 > 0.50), confirming that the data contains learnable signal. DistilBERT achieves the highest macro F1 at 0.580, comparable to the Random Forest variants despite operating on raw text rather than engineered features. The narrow margin between structured-feature and deep-learning approaches suggests that the predictive signal lives primarily in review metadata rather than semantic content.

Figure 1 visualizes this comparison.

The structured Random Forest catches 28% of flagged restaurants (191 of 671 in the test set) at 11% precision, trading 1,548 false positives for 191 true positives. Adding TF-IDF features provides marginal improvement (0.568 $\rightarrow$ 0.570), suggesting that bag-of-words text patterns add little beyond what the 6 engineered features already capture.

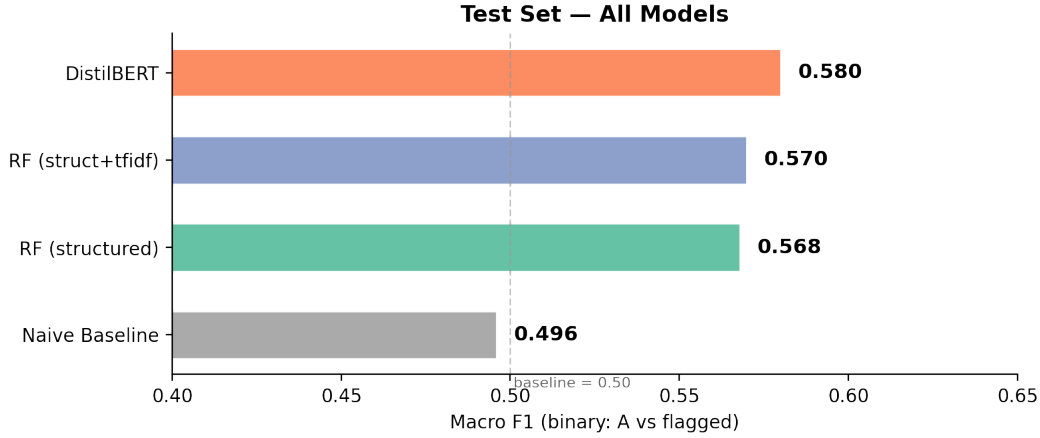


Figure 1: Test-set macro F1 across all four models. The dashed line marks the majority-class baseline at 0.50. All trained models exceed baseline, with DistilBERT (0.580) and RF+TF-IDF (0.570) performing comparably. The narrow margin between structured-feature and deep-learning approaches suggests that the predictive signal lives primarily in review metadata rather than semantic content.

8.2 SHAP Feature Importance

Figure 2 shows the mean absolute SHAP values for each feature in the structured Random Forest.

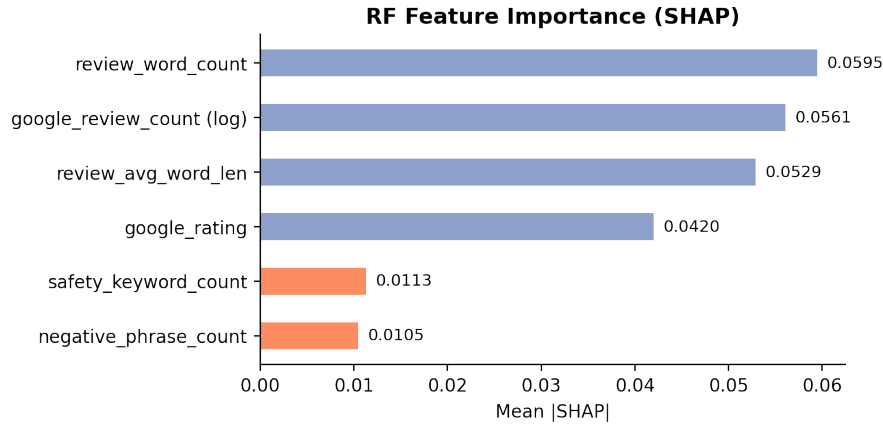


Figure 2: Random Forest feature importance via SHAP. Blue bars indicate review metadata features; orange bars indicate safety-specific text features. Review volume proxies carry the most signal. Safety keyword and negative phrase counts contribute modestly.

The SHAP analysis reveals that the model’s predictive power comes primarily from review volume and length, not from safety-specific language. Google star rating alone is a weak predictor (rank 4). The two explicitly safety-oriented features contribute the least, consistent with the hypothesis that the model learns establishment-type proxies from review metadata, not food safety from review sentiment.

8.3 Threshold Sweep Analysis

The binary A vs. Flagged framing uses NC DHHS’s grade cutoffs ($A \geq 90$, $B = 80\text{--}89$, $C < 80$). We performed a threshold sweep to test whether review features are more predictive at different

Table 5: SHAP feature importance for the structured Random Forest. Review volume features dominate; safety-specific keyword features contribute modestly.

Rank	Feature	Mean SHAP
1	review_word_count	0.0595
2	google_review_count_log	0.0561
3	review_avg_word_len	0.0529
4	google_rating	0.0420
5	safety_keyword_count	0.0113
6	negative_phrase_count	0.0105

inspection score cutoffs, i.e., at what score threshold do reviews begin to carry signal?

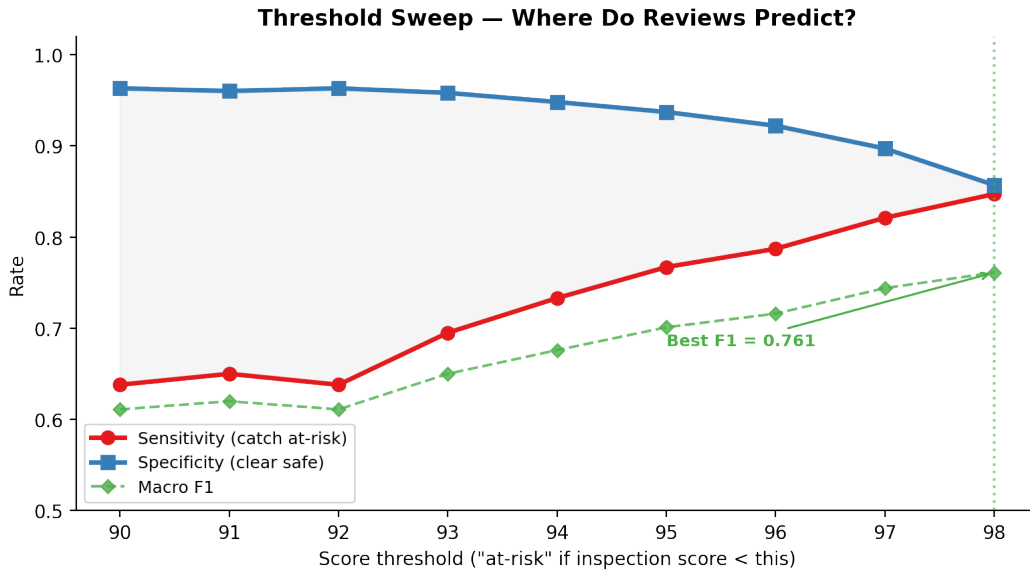


Figure 3: Threshold sweep: sensitivity, specificity, and macro F1 across inspection score thresholds. As the “at-risk” cutoff rises from 90 to 98, sensitivity increases from 64% to 85% while specificity decreases from 96% to 86%. Best macro F1 (0.761) at the <98 threshold.

At the <98 threshold, the model achieves macro F1 = 0.761 with 85% sensitivity, substantially higher than the 0.57 at the A/B/C grade boundary. This finding informed the deployment decision: the web application uses a configurable threshold rather than fixed grade bins, allowing users to adjust sensitivity to their risk tolerance.

8.4 Confusion Matrices

Naive Baseline (test set, $n = 46,232$):

Random Forest (test set, $n = 46,232$):

The RF trades 1,548 false positives for 191 true positives, a precision of 11% at 28% recall. In this domain, catching 191 flagged restaurants at the cost of over-flagging 1,548 safe ones may be acceptable as a screening tool (surface candidates for manual review) but not as a consumer-facing definitive label.

Table 6: Threshold sweep results. At the <98 cutoff, the model achieves macro F1 = 0.761 with 85% sensitivity, substantially higher than 0.57 at the A/B/C grade boundary.

Threshold	Sensitivity	Specificity	Macro F1
< 90 (grade B/C)	0.638	0.963	0.611
< 93	0.695	0.958	0.650
< 95	0.767	0.937	0.701
< 97	0.821	0.897	0.744
< 98	0.847	0.857	0.761

	Pred Safe	Pred Flagged
True Safe	45,561	0
True Flagged	671	0

9 Error Analysis

We examine 5 specific mispredictions from the Random Forest: 4 false negatives (flagged restaurants predicted as safe) and 1 false positive (a safe restaurant the model flagged). Of 671 flagged restaurants in the test set, 480 were missed (28% recall). Of 45,561 safe restaurants, 1,548 were over-flagged (3.4% false positive rate).

9.1 Case 1: La Tovar Mexican Kitchen and Seafood

False Negative, Grade B, Score 86.5

Google rating 4.6 | Safety keywords 0 | $P(\text{flagged}) = 0.496$

“Found another gem here in Greenville! Friendly and helpful staff and the food was great tasting and nicely portioned.”

Root cause: High rating (4.6), zero safety keywords, entirely positive reviews. The model came close to the decision boundary ($P = 0.496$) but the absence of any negative textual signal kept it on the safe side.

Mitigation: Lower the classification threshold from 0.50 to 0.45 for this borderline band, catching near-threshold cases at the cost of more false positives.

9.2 Case 2: El Potrillo Moyock

False Negative, Grade B, Score 86.5

Google rating 4.4 | Safety keywords 0 | $P(\text{flagged}) = 0.483$

“Very disappointing to spend \$70+ and have to go home and cook dinner... the cheese dip was too salty for any of us to eat.”

Root cause: The review expresses dissatisfaction with food quality, not safety. “Disappointing” and “too salty” are dining complaints, not hygiene indicators.

Mitigation: Add a text-derived “complaint specificity” feature distinguishing dining complaints from hygiene-adjacent language. Alternatively, incorporate prior inspection scores; a restaurant with borderline history is more likely to fail again regardless of review sentiment.

9.3 Case 3: El Mexicano Tacos and Tequila

False Negative, Grade B, Score 88.5

Google rating 4.7 | Safety keywords 0 | $P(\text{flagged}) = 0.481$

“Ordered the seafood fajitas and seafood dip and the house made guac and all of it was so delicious.”

	Pred Safe	Pred Flagged
True Safe	44,013	1,548
True Flagged	480	191

Root cause: 4.7-star rating with glowing reviews. The inspection failure is entirely invisible in the customer experience; the gap between what customers observe and what inspectors measure is fundamental to this problem.

Mitigation: Incorporate inspection history as a feature. A restaurant flagged before is more likely to be flagged again.

9.4 Case 4: Hibachi To Go

False Negative, Grade B, Score 88.5

Google rating 3.9 | Safety keywords 0 | Negative phrases 3 | $P(\text{flagged}) = 0.476$

“Food poisoning. I do intermittent fasting so I know it’s from here. I was skeptical of the quality, but I wanted to believe there was a healthier drive thru food option. Never again.”

Root cause: Despite 3 negative phrases (“food poisoning,” “never again,” “disgusting”), the model still predicted safe. The `negative_phrase_count` feature contributes modestly (mean $|\text{SHAP}| = 0.010$, ranked last).

Mitigation: Introduce a binary “explicit illness mention” feature, separate from general negative phrase count. A review mentioning “food poisoning” is qualitatively different from “disappointing.”

9.5 Case 5: EC Pho Vietnamese Noodle House

False Positive, Grade A, Score 90.0

Google rating 4.3 | Safety keywords 0 | Negative phrases 1 | $P(\text{flagged}) = 0.991$

“Way to go Greenville!!! This EC Pho is legit!!! Jonny added to the experience with his excellent customer service!!!”

Root cause: Safe record (score 90.0, grade A) but the model flagged with near-certainty ($P = 0.991$). Balanced class weighting inflates minority-class importance by $68\times$, causing over-flagging of statistically unusual review profiles.

Mitigation: Calibrate probability outputs using Platt scaling or isotonic regression, then choose a threshold appropriate for the deployment context.

9.6 Summary of Root Causes

Table 7: Root causes of the five analyzed mispredictions and their addressability.

Root Cause	Cases	Addressable?
Reviews discuss taste/service, not safety	2, 3	Partially: complaint specificity features + inspection history
Near-threshold borderline cases	1	Yes: threshold tuning
Safety text features underweighted	4	Yes: explicit illness-mention feature
Balanced weighting over-flags unusual profiles	5	Yes: probability calibration

The dominant failure mode is structural: customers write about what they experience (taste, ambiance, service), while inspectors evaluate what customers cannot see (sanitizer concentration, cold-holding temperatures, pest evidence).

10 Experiment: Impact of Data Pipeline Quality on Model Performance

10.1 Hypothesis

We discovered two bugs in the data pipeline that silently degraded the training data. We hypothesized that fixing them would provide enough training signal for the models to learn from the minority class.

10.2 Bug 1: Case-Sensitive Fuzzy Matching

`fuzz.token_sort_ratio` was called without case normalization. NC DHHS records use ALL-CAPS names; Google uses title case. “BOJANGLES” vs. “Bojangles” scored 11.1% instead of 100%. **Fix:** Added `processor=fuzz_utils.default_process`. Usable Google matches jumped from 432 to 14,868 (34× increase).

10.3 Bug 2: BOM-Corrupted Inspection Dates

The DHHS CSV export includes a UTF-8 BOM (byte order mark). The scraper attempted to strip the BOM from the decoded text, but the Python `requests` library decoded the three BOM bytes as literal characters rather than the Unicode codepoint `U+FEFF`. This corrupted the first column header, so lookups for “Inspection Date” returned empty for every row.

With all dates null, deduplication on `(state_id, inspection_date)` treated every inspection of the same restaurant as a duplicate and kept only the first one (typically an A-grade inspection). This collapsed 232K rows to 31K and discarded 94% of B/C grade samples.

Fix: Read CSV from raw bytes with BOM-aware encoding:

```
pd.read_csv(io.BytesIO(resp.content), encoding="utf-8-sig")
```

10.4 Results

Table 8: Impact of the two data pipeline fixes. The BOM fix was the critical change, recovering 200K inspection rows and 17× more flagged training samples.

Metric	Before Fixes	After Fixes	Change
Total inspection rows	31,760	231,160	+7.3×
Flagged samples (B+C)	197	3,354	+17×
Imbalance ratio	160:1	68:1	−58%
Rows with Google reviews	14,529	111,542	+7.7×
RF Macro F1	0.50	0.57	+0.07
RF Flagged Recall	0.00	0.28	+0.28
RF Flagged F1	0.00	0.16	+0.16

10.5 Interpretation

The first fix (case normalization) increased Google data coverage 34× but did not improve model performance on the 31K-row dataset because the bottleneck was not feature coverage but label quality. The second fix (BOM encoding) was critical: it recovered 200K inspection rows and increased flagged training samples from 197 to 3,354, giving the model 17× more minority-class examples.

This demonstrates that **data pipeline quality is a prerequisite for model quality**. Two silent bugs (one in string processing, one in CSV parsing) were sufficient to make a learnable problem appear unlearnable. No amount of architecture engineering or hyperparameter tuning could have compensated for 94% of the minority class being silently discarded.

This experiment validates the decision to deploy the Random Forest as the production model: RF responded to the data quality improvement (macro F1 0.50 $\rightarrow$ 0.57), confirming that structured features carry learnable signal once sufficient training data is available. The threshold sweep (Section 8.3) further demonstrated that the signal strengthens at finer score cutoffs, informing the application’s configurable-threshold design.

11 Interactive Application

The deployed web application (<https://nocapchicken.github.io>) provides a consumer-facing interface for restaurant safety screening. Users search for a restaurant by name; the application queries the Google Places API for review data, runs the Random Forest model on the extracted features, and displays:

1. **Risk assessment:** Binary safe/flagged prediction with probability score.
2. **SHAP explanation:** Per-feature attribution showing which review characteristics contributed to the prediction.
3. **Google metadata:** Star rating, review count, and review excerpts for context.

The application is inference-only (`app/` contains no training code). Feature construction in the inference pipeline (`app/inference.py`) uses the same constants as the training pipeline (`scripts/feature_constants.py`) to prevent feature drift.

12 Conclusions

We investigated whether crowdsourced Google reviews can predict NC restaurant food safety inspection grades. The project produced three main findings:

1. Data pipeline quality determines model quality. Two silent bugs (case-sensitive fuzzy matching and BOM-corrupted inspection dates) made a learnable problem appear unlearnable. Fixing them increased flagged training samples from 197 to 3,354 and moved RF macro F1 from 0.50 to 0.57.

2. Review metadata carries measurable but limited signal. The Random Forest catches 28% of flagged restaurants at 11% precision, finding patterns in review volume and text length that correlate with inspection grades. The signal is primarily proxy-based. DistilBERT achieves comparable performance (macro F1 = 0.58) from raw text, confirming that the signal, whether accessed via engineered features or a language model, is similarly bounded.

3. Threshold choice matters more than model choice. The threshold sweep shows that at a <98 inspection score cutoff, the model achieves macro F1 = 0.76 with 85% sensitivity, substantially higher than the 0.57 at the A/B/C grade boundary. The predictive signal in reviews is strongest for distinguishing near-perfect inspections from any-deduction inspections.

A review-based model can serve as a screening tool (surface high-risk candidates for manual review) but not as a definitive safety label. The 11% precision rate at the A/B/C boundary means 9 out of 10 flagged restaurants are actually safe, which is acceptable for a first-pass filter but not for a consumer-facing warning.

13 Future Work

Given another semester, we would pursue these directions:

1. **Inspection history as features.** A restaurant’s prior scores, violation counts, and time since last inspection are strong predictors. These are available from NC DHHS and would shift the model from review-based prediction to inspection-trend prediction.

2. **Complaint-driven signals.** NC DHHS accepts public complaints that trigger inspections. Mining complaint text (which describes safety concerns directly, unlike reviews that describe dining experience) would provide features with direct domain relevance.
3. **Anomaly detection reframing.** Treat the problem as anomaly detection rather than binary classification: identify restaurants whose review profile is statistically unusual given their inspection history. This sidesteps the class imbalance problem entirely.
4. **Cross-state validation.** NYC has a higher proportion of B and C grades. Testing whether the RF’s signal generalizes to a less imbalanced distribution would clarify whether our finding is specific to NC’s 98.5% A-grade rate.
5. **DistilBERT with focal loss.** The current DistilBERT results (macro F1 = 0.58) use class-weighted cross-entropy. Focal loss, which further downweights easy examples, may improve recall on the hardest flagged cases.
6. **Review-level classification.** A sentence-level classifier could label individual review sentences as safety-relevant or not, then use the proportion as a structured feature for the RF.

14 Commercial Viability

A model that reliably predicts food safety from publicly available data would have clear commercial value: integration into restaurant discovery platforms, insurance underwriting for food service businesses, and public health surveillance tools.

However, our results show that review data alone is insufficient. A commercially viable product would need to combine review analysis with (1) public inspection records, (2) complaint data, and (3) operational signals (e.g., staff turnover, hours of operation changes). The review component would serve as one input among many, not as the sole predictor.

The web application (<https://nocapchicken.github.io>) demonstrates the UX pattern: search for a restaurant, see its Google rating alongside a model-generated risk assessment with SHAP explanations. The interface is production-ready; the underlying model needs richer data sources to be commercially useful.

15 Ethics Statement

Data provenance. NC DHHS inspection records are public government records under NC Public Records Law (G.S. 132-1). Google Places data was accessed via the official API under Google’s Terms of Service. No personally identifiable information about restaurant patrons was collected or used.

Potential harms. A false-positive prediction (labeling a safe restaurant as flagged) could cause reputational harm to a business. A false-negative (labeling an unsafe restaurant as safe) could lead consumers to eat at establishments with sanitation violations. Our current model produces 1,548 false positives and 480 false negatives on the test set (11% precision, 28% recall). The low precision means the model should be used as a screening tool, not a definitive label.

Limitations of inference. An inspection grade reflects a point-in-time assessment. Conditions can change between inspections. Users should be reminded that model predictions are not substitutes for official health inspection records.

Bias considerations. The fuzzy name matching pipeline may systematically fail to link restaurants with non-English names, leading to lower Google data coverage for certain cuisines. We did not audit for this bias but acknowledge it as a limitation.

Acknowledgments

This project was completed as part of AIPI 540 (Deep Learning Applications) at Duke University. We thank the NC Department of Health and Human Services for making inspection records publicly available.

References

- Kristen M Altenburger and Daniel E Ho. Predicting restaurant health inspection outcomes from online reviews. In *The World Wide Web Conference (WWW)*, pages 1–11. ACM, 2019.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- Jun Seok Kang, Polina Kuznetsova, Michael Luca, and Yejin Choi. Using Yelp reviews to improve restaurant inspection forecasting. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, 2013.
- Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- Elaine O Nsoesie, David L Buckeridge, and John S Brownstein. Online reports of foodborne illness capture foods implicated in official foodborne outbreak reports. *Preventive Medicine*, 67:264–269, 2014. doi: 10.1016/j.ypmed.2014.08.003.
- Adam Sadilek, Henry Kautz, Lauren DiPrete, Brian Labus, Eric Portman, Jack Teber, and Vincent Silenzio. Machine-learned epidemiology: real-time detection of foodborne illness at scale. *npj Digital Medicine*, 1(1):36, 2018. doi: 10.1038/s41746-018-0045-1.
- Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.